

**DEPARTMENT OF COMPUTER & INFORMATION SYSTEMS ENGINEERING
BACHELORS IN COMPUTER SYSTEMS ENGINEERING**

Course Code: CS-116

Course Title: Object Oriented Programming

Complex Engineering Problem

FE Batch 2025, Spring Semester 2026

Grading Rubric

TERM PROJECT

Student No.	Name	Roll No.
S1	Zainab Khan	CS-25014
S2	Horia Younus	CS-25016
S3	Zehra Baig	CS-25017
S4	Abeer Ahmed Siddiqui	CS-25018

CRITERIA AND SCALES					Marks Obtained			
					S1	S2	S3	S4
Criterion 1: Does the application meet the desired specifications and produce the desired outputs? (CPA-1, CPA-3)								
1	2	3	4					
The application does not meet the desired specifications and is producing incorrect outputs.	The application partially meets the desired specifications and is producing incorrect or partially correct outputs.	The application meets the desired specifications but is producing incorrect or partially correct outputs.	The application meets all the desired specifications and is producing correct outputs.					
Criterion 2: How well is the code organization?								
1	2	3	4					
The code is poorly organized and very difficult to read.	The code is readable only to someone who knows what it is supposed to be doing.	Some part of the code is well organized, while some part is difficult to follow.	The code is well organized and very easy to follow.					
Criterion 3: How friendly is the application interface? (CPA-1, CPA-3)								
1	2	3	4					
The application interface is difficult to understand and use.	The application interface is easy to understand and but not that comfortable to use.	The application interface is very easy to understand and use.	The application interface is very interesting/ innovative and easy to understand and use.					
Criterion 3: How friendly is the application interface? (CPA-1, CPA-3)								
1	2	3	4					
The application interface is difficult to understand and use.	The application interface is easy to understand and but not that comfortable to use.	The application interface is very easy to understand and use.	The application interface is very interesting/ innovative and easy to understand and use.					
Criterion 4: How does the student performed individually and as a team member? (CPA-2, CPA-3)								
1	2	3	4					
The student did not work on the assigned task.	The student worked on the assigned task, and accomplished goals partially.	The student worked on the assigned task, and accomplished goals satisfactorily.	The student worked on the assigned task, and accomplished goals beyond expectations.					
Criterion 5: Does the report adhere to the given format and requirements?								
1	2	3	4					
The report does not contain the required information and is formatted poorly.	The report contains the required information only partially but is formatted well.	The report contains all the required information but is formatted poorly.	The report contains all the required information and completely adheres to the given format.					
Total Marks:								

COMPLEX ENGINEERING PROJECT REPORT



**NED University Of Engineering
And Technology
Computer And Information
Systems Engineering Department**

**Object Oriented Programming - CS-116
Group ID - G1-3**

**Instructor - Miss Tehreem Khan
Dated : May 16th 2026**

THE TEAM

Batch 2025
Spring Semester 2026
Section A



Zainab Khan
CS 25014



Horia Younus
CS 25016



Zehra Baig
CS 25017



Abeer Ahmed Siddiqui
CS 25018



INTRODUCTION TO THE PROJECT

- Introduction
- Project Description
- Project Scope
- Technologies Used

INTRODUCTION

Libraries are one of the most essential knowledge-sharing institutions in any academic or community setting. However, traditional paper-based library management practices are slow, error-prone, and difficult to scale. As the volume of resources grows and user expectations rise, there is a clear need for a digital solution that can automate routine library operations while remaining easy to use for both members and administrators.

The Hybrid Library Management System (LMS) is a console-based C++ application developed to address this need. It models real-world library operations through a carefully designed class hierarchy, persistent file-based storage, and a role-based user interface. The system supports two distinct categories of resources — physical items such as books and journals, and online digital resources — treating each with its own borrowing and return policies. The application is built entirely using the principles of Object-Oriented Programming (OOP), demonstrating inheritance, abstraction, polymorphism, encapsulation, operator overloading, and structured exception handling.

TECHNOLOGY OVERVIEW

Component	Detail
Language	C++ (C++17 standard)
Paradigm	Object-Oriented Programming (OOP)
OOP Concepts Used	Inheritance, Polymorphism, Abstraction, Encapsulation
Persistence	File-based storage (.txt files)
Interface	Console / Command-Line (menu-driven)
Platform	Cross-platform (Windows executable provided)
Build Tool	VS Code Tasks (tasks.json)

SCOPE

The scope of this project encompasses the full lifecycle management of library resources within a single institution. The system handles up to 1,000 physical resources and an unbounded number of online resources, stored and managed in-memory with file-backed persistence. It supports member self-service operations as well as comprehensive administrative controls.

The system is intentionally designed as a standalone console application without a graphical user interface or networked database backend, making it suitable as an academic demonstration of C++ OOP concepts. Extensions such as a GUI, networked access, or SQL-based persistence are outside the current scope but represent natural directions for future development.

PROBLEM DESCRIPTION

Public and academic libraries routinely face a set of interrelated operational challenges that a manual or paper-based system cannot adequately address:

- **Resource visibility:**

Members have no reliable way to determine in real time which resources are currently available for borrowing.

- **Borrow tracking:**

Keeping accurate records of who borrowed which resource, on what date, and when it is due for return is cumbersome when done manually.

- **Fine enforcement:**

Calculating and applying late-return fines consistently across all members requires automated date arithmetic and per-member balance management.

- **Dual resource management:**

Physical items and online digital resources have fundamentally different borrowing rules, access charges, and return policies that must be handled distinctly yet managed under a unified interface.

- **Administrative oversight:**

Administrators need a centralized view of resource inventory, member borrow histories, and currently outstanding items to make effective management decisions.

- **Data persistence:**

All records — member accounts, resource catalogues, and borrow histories — must survive program restarts without the use of a database engine.

The goal of this project is to design and implement a modular, extensible C++ application that resolves each of the above challenges through well-structured OOP design patterns and file-based data persistence.



PROJECT FEATURES

- Objectives
- Distinguishing Features
- Implementation Files
- Design Constraints

PROJECT OBJECTIVES

The Hybrid LMS is designed to satisfy the following specific objectives:

- Maintain a searchable catalogue of both physical and online library resources, displaying only available items to borrowing members.
- Provide secure, account-based member registration and login, with personal balance management and borrowing history.
- Enforce a maximum concurrent borrow limit of two resources per member at any point in time.
- Automatically compute and deduct late-return fines for physical resources based on the number of overdue days.
- Compute proportional access-charge refunds for online resources returned before their 14-day access window expires.
- Allow administrators to add, remove, and issue physical resources, process returns, and generate comprehensive reports.
- Persist all data — members, resources, and borrow records — to plain-text CSV files so that the application state is fully restored on every startup.
- Demonstrate at least six distinct OOP features through a clean, well-commented, multi-file codebase.

DESIGN CONSTRAINTS AND ASSUMPTIONS

- The application targets the C++17 standard and relies solely on the C++ Standard Library — no third-party libraries or database engines are used.
- Data persisted in plain-text CSV files (members.txt, PhysicalResources.txt, OnlineResources.txt, Borrow Record Storage.txt). All file paths are relative to the working directory.
- Physical resources are managed through a raw pointer array of fixed capacity (1,000 entries) to demonstrate manual dynamic memory allocation and deallocation. Online resources are managed via `std::vector<std::unique_ptr>` to demonstrate modern C++ memory safety.
- The BorrowRecord class uses a static integer counter (`IDCount_`) to auto-generate sequential borrow IDs. This counter persists only for the lifetime of a single program run; upon reload, IDs are reconstructed from file data.
- Date arithmetic uses a simplified model (30 days per month, 365 days per year) sufficient for fine and refund calculations within the scope of this application.
- A member may hold at most two simultaneously active borrow records (`maxBorrowLimit_ = 2`). Physical resource due dates are set to two days from issuance; online resources have a 14-day access window.
- The single Administrator account is hard coded at startup (username: admin, password: admin123). Member accounts are fully dynamic, and file backed.
- The user interface is entirely text-based (console/terminal). All menus are presented as numbered options read from standard input.

PROJECT IMPLEMENTATION FILES

The project follows a one-class-per-header convention with corresponding implementation files:

File Name	Responsibility / Description
main.cpp	Entry point: instantiates LMS and calls run()
lms.hpp / lms.cpp	Top-level orchestrator: composes all four manager objects and drives the menu system
Date.hpp / Date.cpp	Date value type with arithmetic, validation, and stream operators
LibraryResource.hpp / .cpp	Abstract base class and shared resource logic
PhysicalResource.hpp / .cpp	Concrete physical resource with fine and damage-charge logic
OnlineResource.hpp / .cpp	Concrete online resource with access-charge and refund logic
BorrowRecord.hpp / .cpp	Borrow transaction record entity class
UserManagement.hpp / .cpp	User, Member, and Administrator class definitions and implementations
*Manager.hpp / *Manager.cpp (x4)	Manager classes for borrows physical resources, online resources, and users
*Storage.hpp / *Storage.cpp (x4)	Storage classes handling CSV serialisation and deserialisation for each entity type
*.txt (x4 data files)	Persistent data files for members, physical resources, online resources, and borrow records

DISTINGUISHING FEATURES

Hybrid collection support:

Manages both physical resources (issued/returned administratively) and online resources (borrowed/returned by members) within one system, with consistent IDs (PR-... vs OR-...) and shared borrowing records.

Role-based workflows:

Separate, purpose-built flows for Members (self-service account management and online borrowing) and Administrators (catalog management, physical issuance/returns, and reporting).

Persistent storage architecture:

Uses dedicated storage components to load/save members, resources, and borrow records to text files, so state is preserved across runs without requiring a database.

Borrow-record lifecycle tracking:

Each transaction captures memberID, resourceID, issue/due/return dates, and return status, enabling traceability and accurate history reporting.

Policy-driven returns (charges/refunds):

Return processing calls a resource-specific return policy (applyReturnPolicy)—physical returns can apply charges, while online returns can calculate refunds for unused access days.

Operational constraints and validation:

Enforces borrowing limits, checks availability, validates IDs/formats, and (for online borrowing) verifies sufficient member balance before issuing access.



OOP CONCEPT IMPLEMENTATION

- Inheritance
- Encapsulation
- Polymorphism
- Abstraction
- File Handling
- Exception Handling
- Smart Pointers

OOP CONCEPTS IMPLEMENTATION:

Encapsulation:

Overview:

Encapsulation refers to the bundling of data members and functions within classes and restricting direct access through private / protected access specifiers.

Implementation:

- All attributes were considered private to restrict direct access from child objects.
- The private data members were assigned necessary setters and getters for data modification and readability.
- All the public member functions allowed for controlled access to the class.

Abstraction:

Overview:

Abstraction in OOP is the concept of hiding complex implementation details and showing only the essential features of an object.

Implementation:

- **Members** interact with only the necessary features such as searching for resources and issuing or returning them, without knowing how these operations are handled internally.
- The complex processes like database updates, availability checks, and record management are hidden from the **Member**. While the **Administrator** has access to the database and management.
- Member class exposes simple functions like `borrowOnlineResource()` and `returnOnlineResource()` to the menu.
- Internal logic like validating borrow limit, checking balance, coordinating with managers, and deducting charges is completely hidden from the user interface layer.

Inheritance:

Overview:

Inheritance in OOP allows one class to acquire the properties and behaviors of another class.

Implementation:

- **LibraryResource** which is inherited by its child classes **PhysicalResource** and **OnlineResource** who share the common features of basic properties such as title , author , genre etc.
- User is the abstract base class holding shared fields like `username_`, `password_`, `firstName_`, `lastName_`.
- Member inherits User and adds borrow-specific behaviour like `balance_`, `borrowCount_`, and borrow history.
- Administrator inherits User and adds resource issuance and return processing capabilities.

Polymorphism:

Overview:

Polymorphism allows functions to behave differently based on the object that calls them. It is implemented using virtual functions.

Implementation:

- **LibraryResource** includes a virtual function **ApplyReturnPolicy** which has separate functionalities in both **PhysicalResources** and **OnlineResources**.
- **Administrator::processReturnOfPhysicalResource()** calls **resource->applyReturnPolicy(record)** through a **LibraryResource*** pointer without knowing the actual type at compile time

Smart Pointers:

Overview:

Smart pointers are special C++ pointers that automatically manage dynamic memory. They help prevent memory leaks by automatically deleting objects when they are no longer needed.

Implementation:

- **OnlineResourceManager** uses a vector of smart pointers that point to an object of **OnlineResources**.
- It allows automatic deletion of objects when the vector pointer is removed via **Administrator** controls.

Exception Handling:

Overview:

Exception handling is a mechanism used in programming to detect and manage runtime errors without abruptly terminating the program. In C++, it is implemented using try, throw, and catch blocks, where risky code is placed inside a try block, errors are generated using throw, and handled in a catch block.

Implementation:

- Implemented exception handling using try-catch blocks to safely handle invalid numeric conversions while extracting IDs and reading stored file data in **Physical Manager**.
- Added validation for corrupted or improperly formatted file records, allowing the system to skip errors and continue execution safely for example, in **Physical** and **Online Storage**.
- **setPassword()** throws **std::invalid_argument** if the password is too short.
- Caught inside **updatePassword()** using try-catch — error message is displayed cleanly and old password remains unchanged, preventing corruption of member credentials.

File Handling:

Overview:

File handling is the process of storing, reading, updating, and managing data in files using a programming language like C++. It allows programs to save information permanently on disk using file streams such as `ifstream` for reading, `ofstream` for writing, and `fstream` for both operations. File handling is commonly used to store user data, logs, or system records so that information is not lost when the program ends.

Implementation:

- Implemented file handling using `ifstream` and `ofstream` to load and save physical resource data for persistent storage in **member** and **borrow record storage**.
- `MemberStorage` serialises the entire `vector<Member>` to `members.txt` in CSV format after every change — registration, balance update, password change.
- `loadAllMembers()` reads it back on startup using `stringstream` with comma delimiter, reconstructing all `Member` objects into memory.
- Designed CSV reading and writing mechanisms to ensure consistent data transfer between stored files like in **physical storage** and **online storage**.

Composition:

Overview:

Composition is an object-oriented programming concept where a class contains objects of other classes as part of its data members, forming a “has-a” relationship. It allows complex systems to be built by combining simpler classes, improving modularity and code reuse. In C++, composition is implemented by including objects of one class inside another, meaning the lifetime of the contained objects is usually managed by the containing class.

Implementation:

- Implemented composition by integrating related objects within classes to model strong “has-a” relationships between system components for example relation between **OnlineManager** and **OnlineStorage**.
- **PhysicalManager** contains object of **PhysicalStorage** implementing composition.
- `User_Manager` contains a `MemberStorage` object directly as a data member — `User_Manager` has-a `MemberStorage`.
- Lifetime of `MemberStorage` is fully tied to `User_Manager` — created and destroyed with it.



SYSTEM OVERVIEW

- Interfaces
- System Flow
- UML Diagram

SYSTEM OVERVIEW:

Interfaces:

User Login Interface:

```
===== WELCOME TO HYBRID LIBRARY MANAGEMENT SYSTEM =====

      " Nothing Is Pleasanter Than Exploring A Library "
                        - Walter Savage Landor

1. Member Login
2. Create Member Account
3. Admin Login
0. Exit
=====
```

Feature	Description
Account Creation	A new user provides a username, password, first/last name, address, and initial balance. A unique numeric member ID is auto assigned.
Secure Login	Credentials are verified against the in-memory member list loaded from members.txt. Incorrect credentials are rejected gracefully.

Admin Interface:

```
===== ADMIN MENU =====

1. Add Physical Resource
2. Add Online Resource
3. Issue Physical Resource
4. Process Return of Physical Resource
5. Remove Physical Resource
6. Remove Online Resource
7. Generate Reports
0. Logout
=====
```

Feature	Description
Add Resource	Administrator can add a new physical or online resource. A unique PR-n or OR-n identifier is auto-generated.
Remove Resource	Administrator can remove a resource from the catalogue by its resource ID.

Issue Physical Resource	Administrator selects a resource ID and a target member ID. The system verifies availability, creates a BorrowRecord, and updates the member borrow count.
Process Physical Return	Administrator provides a borrow ID. The system records the return date, prompts for resource condition, calculates late fines and damage charges, and displays the total amount owed.
Borrow History Report	Prints a formatted table of all borrow records in the system.
Issued Resources Report	Prints all currently issued physical or online resources filtered by availability status.

Report Interface:

```

===== REPORT GENERATOR =====
1. Print All Customers' Borrow History Report
2. Print Currently Issued Physical Resources' Report
3. Print Currently Issued Online Resources' Report
0. Back
=====

```

Feature	Description
Print All Customers Borrow History Report	Displays a formatted and tabular collection of the borrow history of all the customers and their transactions
Print Currently Issued Resources Report	Displays a formatted and tabular collection of the respective resources that haven't been returned.

Member Interface:

```
===== MEMBER MENU =====

    " A Reader Lives A Thousand Lives Before He Dies ,
      A Man Who Never Reads Lives Only One. "
                                - George R. R. Martin

Welcome Fatima Ahmed
1. Manage My Account
2. Physical Resource Interface
3. Online Resource Interface
0. Logout
=====
```

Feature	Description
Account Management	Members can view a tabular borrow history showing borrow ID, resource ID, issue date, due date, return date, and return status.
Resources Interfaces	Members can deposit funds to their account balance from the account management menu.

Account Management Interface:

```
===== MANAGE MY ACCOUNT =====

1. Update Personal Details
2. Update Balance (Deposit)
3. Update Password
4. View Borrow History
0. Back
=====
```

Feature	Description
Update Details	Members can update personal details such as Name , Address etc.
Update Password	Members can update their password from this menu.
Update Balance	Members can deposit funds to their account balance from the account management menu.
View Borrow History	Members can view a tabular borrow history showing borrow ID, resource ID, issue date, due date, return date, and return status.

Physical Resource Interface:

```
===== PHYSICAL RESOURCE MENU =====
1. Search Physical Resource by ID
2. Search Physical Resource by Title
3. Search Physical Resource by Author
4. Search Physical Resource by Genre
5. View All Available Physical Resources
6. View All Physical Resources
0. Back
=====
```

Feature	Description
Search Functions	Searches and displays the physical resource on the basis of filtering attribute.
View Functions	Displays all the physical resources in tabular form filtered by availability status.

Online Resource Interface:

```
===== ONLINE RESOURCE MENU =====
1. Borrow Resource
2. Return Resource
3. Search Online Resource by ID
4. Search Online Resource by Title
5. Search Online Resource by Author
6. Search Online Resource by Genre
7. View All Available Resources
8. View All Online Resources
0. Back
=====
```

Feature	Description
Search Functions	Searches and displays the online resource on the basis of filtering attribute.
Borrow / Return Resources	Allows for automatic Balance update and borrowing and returning of an online resource.
View Functions	Displays all the online resources in tabular form filtered by availability status.

System Flow:

- **Startup and initialization:**

The program starts by creating a single system controller (LMS) which constructs the core manager modules for users, physical resources, online resources, and borrow records; each manager loads its previously saved data into memory from its corresponding text-file storage (members from members.txt, borrow records from Borrow Record Storage.txt, physical catalog from PhysicalResources.txt, and online catalog from OnlineResources.txt).

- **Main interaction loop:**

The system runs a continuous, menu-driven loop where the user selects one of four options: Member Login, Create Member Account, Admin Login, or Exit. Input is validated (especially numeric choices) to prevent invalid entries from breaking the flow.

- **Member authentication and session:**

On Member Login, credentials are verified against the in-memory member list; if valid, the system enters a dedicated member session loop with three functional branches: Manage My Account, Physical Resource Interface, and Online Resource Interface (plus logout).

- **Member account management:**

The member can update personal details, deposit funds (balance update), change password, and view borrowing history; after any account modification, the updated member data is saved back to persistent storage to maintain consistency across runs.

- **Physical resource browsing (member-side):**

The member can search physical resources by ID/title/author/genre and list available or all resources; these actions retrieve the latest catalog state from storage and present results in a readable format.

- **Online resource usage (member-side):**

The member can borrow and return online resources and also search/list the online catalog. Borrowing enforces key constraints (e.g., a maximum concurrent borrow limit and sufficient account balance to cover the access charge over the access period), then generates a new BorrowRecord with issue and due dates and registers it through the borrow-record manager. Returning an online resource updates the corresponding borrow record (marking it returned and setting a return date) and applies the online resource return policy to compute any eligible refund, which is then credited back to the member balance.

- **New member registration:**

On Create Member Account, the system collects required details, creates the member object, assigns a unique member ID, and persists the new account immediately so the user can log in in future sessions.

- **Administrator authentication and session:**

On Admin Login, the administrator is authenticated (using the system's configured admin credentials), then enters an admin session loop that supports catalog administration, issuing/return processing for physical resources, and reporting.

- **Administrative operations:**

The admin can add/remove physical and online resources (with automatic ID generation), issue physical resources to a member (validating member existence, borrow limit, and resource availability), and process physical returns by updating the borrow record and applying the physical return policy to determine any charges.

- **Report generation:**

The admin can generate system reports such as complete borrow history across members and listings of currently available/issued resources, providing a consolidated operational view for oversight.

- **Shutdown and persistence:**

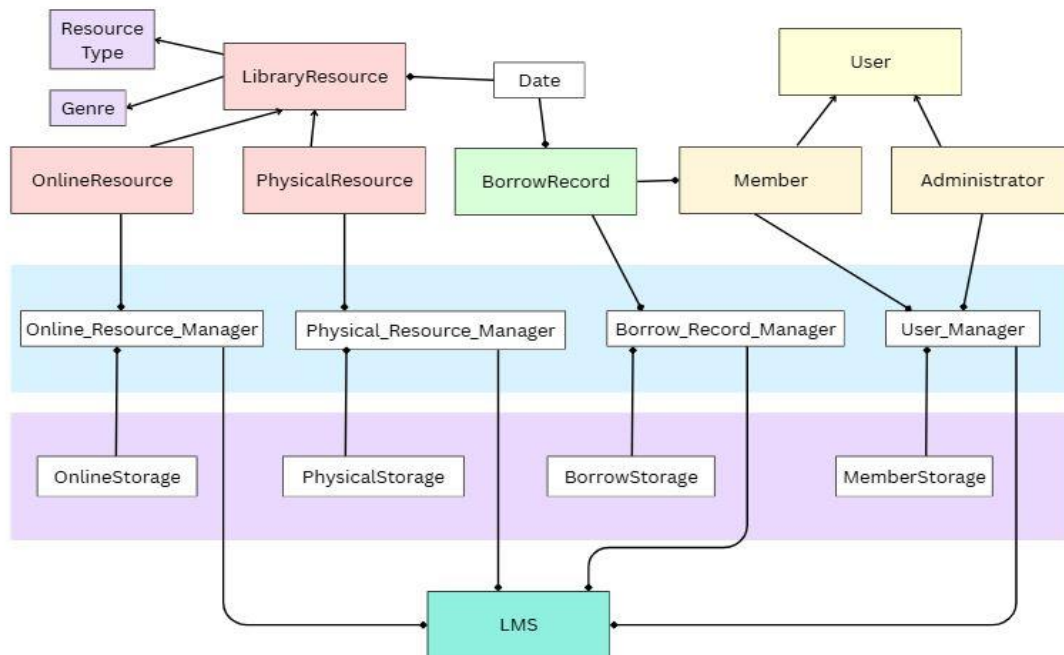
When the user exits, the session loops terminate and manager destructors finalize persistence by writing in-memory state back to the storage files, ensuring that user accounts, resource catalogs, and borrow records remain available for subsequent program executions.

UML Diagram:

Links:

[UML DIAGRAM EMBED](#)

Quick overview of the classes:





SYSTEM ARCHITECTURE

- Domain Layer
- Manager Layer
- Storage Layer
- Utility Classes
- Top Level Orchestrator
Layer

SYSTEM ARCHITECTURE AND CLASS DESIGN

Overview

The application is structured into three principal architectural layers: the domain model (entity classes), the storage layer (file I/O classes), and the manager layer (business-logic classes). A top-level orchestrator class, `lms`, composes one instance of each manager and drives the interactive menu system. This separation of concerns keeps each class focused on a single responsibility, which simplifies testing and future extension.

DOMAIN MODEL (ENTITTY CLASSES):

Library Resource:

The **LibraryResource** class is an abstract base class that represents a general library item in the system. It stores common attributes shared by all resources, such as title, author, publication date, availability status, unique resource ID, genre, and resource type. Since it is designed as a base class, it provides shared functionality and enforces a common interface for all derived classes.

It defines a pure virtual function **applyReturnPolicy**, which has a different implementation in both derived classes ; **OnlineResource** and **PhysicalResource**, allowing each type of resource to have its own return policy logic. This design supports polymorphism and ensures that different resource types can be handled uniformly while still allowing specialized behaviour.

- **OnlineResource Class:**

The **OnlineResource** class is a derived class of **LibraryResource** that represents digital or online materials in the library system. It extends the base functionality by introducing an access charge for using online resources and provides methods to handle payment deduction from a member's account. The class also manages borrowing restrictions and defines a specific return policy for online resources through method overriding.

Each **OnlineResource** Object is assigned a unique ID for example ("OR-1","OR-2"...).

- **PhysicalResource Class:**

The **PhysicalResource** class is a derived class of **LibraryResource** that represents physical items in the library such as books and journals. It extends the base class by introducing a late fine system and a damage charge system to **manage** overdue returns and item condition issues. The class provides functionality to calculate fines based on return delays, determine damage costs based on the condition of the resource, and apply a specific return policy that updates a member's balance accordingly.

Each **PhysicalResource** Object is assigned a unique ID for example ("PR-1","PR-2"...).

User:

The main purpose of this class is to manage all activities related to library members, including account creation, borrowing information, balances, and member records. Each member is assigned a unique ID using automatic ID generation, ensuring proper identification within the system.

The class also maintains a borrow history structure for storing member-related borrowing activity. It manages the overall interaction between the member and the library system while ensuring that borrowing rules and account restrictions are properly enforced.

- **Member Class:**

Member inherits from User and represents a library card holder. A member can borrow and return resources, manage their account balance, and view their borrow history. Each member gets a unique auto-generated ID and can borrow a maximum of two resources at a time.

- **Administrator Class:**

Administrator is a privileged user who inherits from **User**; issues/processes return of physical resources, manages all resources and generates administrative reports. Only one Administrator object exists in the entire system — stored directly inside **User Manager**. **Administrator** has no extra attributes beyond what **User** provides.

BorrowRecord Class:

The **BorrowRecord** class represents one loan transaction in the library system, storing a unique auto-incremented borrowID_ generated from a static counter, the borrowing memberID_, the resourceID_ of the item being borrowed, and the key dates that define the loan lifecycle: issueDate_, dueDate_, and returnDate_. It also tracks whether the item has been returned through returnStatus_. The class is designed mainly as a data holder with read-only accessors for all core fields, while only the return-related data can be updated after creation through setReturnStatus and setReturnDate, which makes it well-suited for accurately recording and managing borrow history without allowing unrelated record fields to be changed.

THE STORAGE LAYER :

Member Storage Class:

Parses and writes member records in CSV format to `members.txt`.

BorrowRecord Storage Class:

Parses and writes borrow records in CSV format to `Borrow Record Storage.txt`

Online Resource Storage Class:

Parses and writes physical resource records in CSV format to `PhysicalResources.txt`.

Physical Resource Storage Class:

Parses and writes borrow records in CSV format to `Borrow Record Storage.txt`

THE MANAGER LAYER :

User Manager Class:

Maintains the Member vector and single Administrator instance; handles login and member CRUD.

BorrowRecord Manager Class:

Manages the in-memory vector of BorrowRecord objects; delegates file I/O to BorrowStorage

Online Resource Manager Class:

Manages the smart-pointer vector of OnlineResource objects; delegates file I/O to OnlineResourceStorage

Physical Resource Manager Class:

Manages the raw-pointer array of PhysicalResource objects; delegates file I/O to PhysicalResourceStorage

UTILILTY CLASSES :

Date Class:

A Date class is a user-defined class in programming that represents a calendar date using attributes such as day, month, and year. It is used to store, manage, and manipulate date-related information. A Date class may also include functions for displaying dates, validating input dates, comparing two dates, or performing operations like calculating the next day or difference between dates.

Genre Enumeration Class:

An enum for genres is used to represent a fixed set of book or media genres with meaningful names instead of numeric values. It improves code readability, organization, and type safety by limiting genre values to predefined options such as Literature , Science , Arts , Technology and Education.

Resource Type Enumeration Class:

An enum for ResourceType is used to represent a fixed set of book or media genres with meaningful names instead of numeric values. It improves code readability, organization, and type safety by limiting genre values to predefined options such as Book , Journal , Reference , Multimedia and GreyLiterature.

TOP-LEVEL ORCHESTRATOR CLASS

Library Management System Class

The **lms** class functions as the top-level controller (orchestrator) for the Hybrid Library Management System, coordinating the main application loop through its *run()* method and providing a structured, menu-driven command-line interface. It composes and orchestrates the core subsystems—online resource management, physical resource management, borrow-

record management, and user management—by routing validated user input to the appropriate manager operations for member and administrator workflows, while relying on those managers' storage mechanisms to preserve system state (accounts, resource catalogs, and transactions) across program executions.



CLASS RELATIONSHIPS

- Inheritance
- Composition
- Association

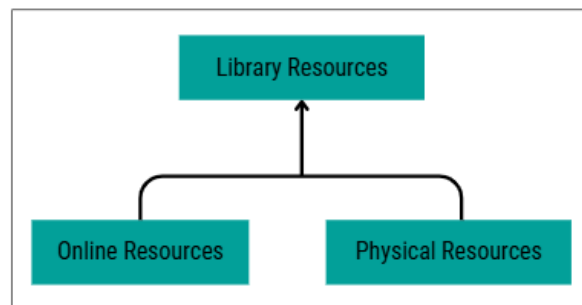
CLASS RELATIONSHIPS

Inheritance:

Hierarchical inheritance is a type of inheritance in OOP where multiple child classes inherit from a single parent class. It allows different classes to share common properties and methods while still having their own specific behaviors. This helps in code reuse and creates a clear class structure.

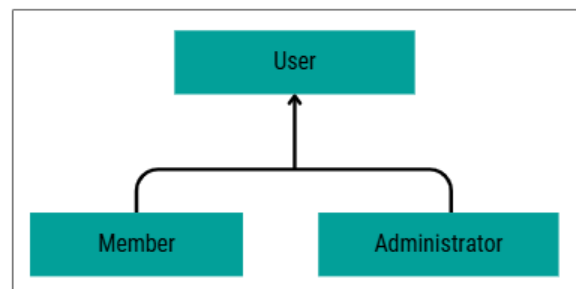
1. Library Resources and Physical / Online Resources:

The system uses hierarchical inheritance, where **LibraryResource** acts as the base class containing common attributes and behaviour shared by all resources. The classes **OnlineResource** and **PhysicalResource** inherit from it, allowing them to reuse common functionality while adding their own specialized features such as access charges and fine systems. This structure promotes code reusability, reduces duplication, and enables polymorphism through overridden functions like **applyReturnPolicy()**.



2. User and Member / Administrator Classes :

This is also a hierarchical inheritance structure, where the **User** class serves as the base class containing common attributes and functionalities shared by all users of the system. The **Member** and **Administrator** classes inherit from **User**, allowing them to reuse shared features while implementing their own specialized roles and permissions. This design supports role-based behavior, where **Member** handles borrowing-related operations and **Administrator** manages system-level control and administration tasks.

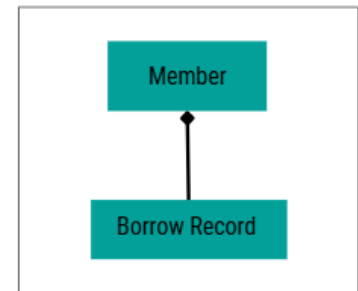


Composition:

Composition in OOP is a design principle where one class contains objects of other classes, showing a “has-a” relationship. It represents strong ownership, meaning the child object depends on the parent object for its existence. If the main object is destroyed, the composed objects are also typically destroyed.

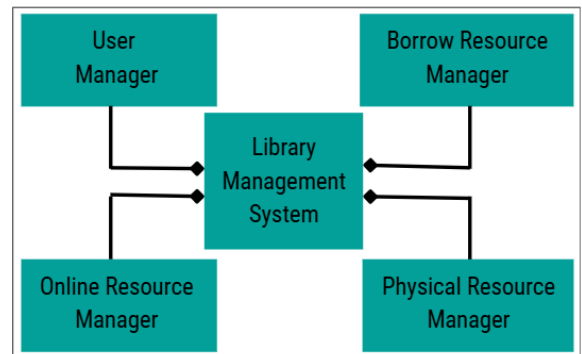
1. Borrow Record and Member Class

In a Library Management System, **Member** and **BorrowRecord** have a **composition relationship**. A Member “owns” a **BorrowRecord**, meaning each borrow record is strongly dependent on the member and cannot exist independently in the system. If a **member** is removed, their associated **borrow records** are also no longer meaningful.



2. Library Management System and Manager Classes:

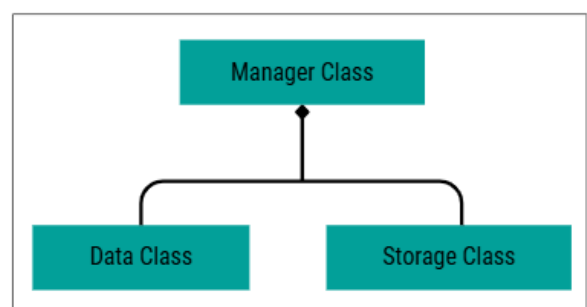
The **Library Management System** is composed of four core classes: **OnlineResource**, **PhysicalResource**, **BorrowRecord**, and **User** (Member/Admin). These classes collectively represent the main functional components of the system, where resources are managed, users interact with the system and borrow records track transactions. Together, they form the backbone of the system’s operations, with each class managing a specific responsibility within the overall design.



3. Manager, Storage and Data Classes:

In this project, composition exists between the **Manager**, **Storage**, and **Data** classes to represent a strong ownership relationship. The **Manager** class is responsible for controlling and maintaining both the storage system and the data it manages, meaning these components are tightly connected to its lifecycles.

The **Manager** Class acts as a database while the **Storage** Classe deal with the filing of their **Data** Class. The system has a total of four such combinations which are listed below:



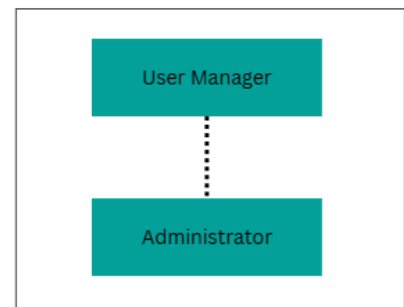
Manager Classes	Storage Class	Data Class
Online Resource Manager	OnlineResourceStorage	OnlineResource
Physical Resource Manager	PhysicalResourceStorage	PhysicalResource
User Manager	MemberStorage	Member
Borrow Record Manager	BorrowStorage	BorrowRecord

Association:

Association in OOP describes a "uses-a" relationship between classes where one class interacts with another without owning it. The associated objects have **independent lifecycles** and neither creates nor destroys the other. In UML, a plain association is represented by a **solid line**, often with a directional arrowhead to indicate which class uses the other.

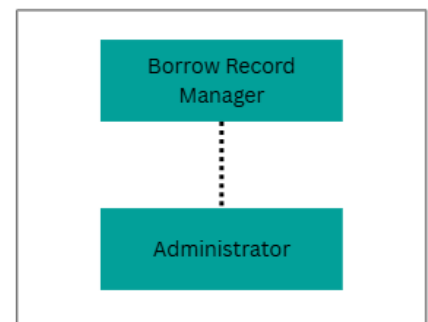
1. Administrator and User Manager:

The **Administrator** class uses **User_Manager** to access member data during administrative operations. Specifically, **issuePhysicalResource()** calls **userMgr.getMemberById()** and **userMgr.saveMembers()** to look up a member and persist the updated borrow count. The **Administrator** does not own or manage the **User_Manager** — it receives it as a reference parameter. This relationship is marked "uses" in the UML.



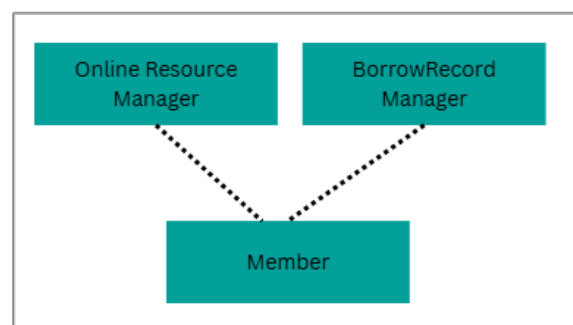
2. Administrator and Borrow Record Manager:

The **Administrator** class interacts with **Borrow Record Manager** when processing returns of physical resources. The method **processReturnOfPhysicalResource()** calls **borrowMgr.updateBorrowRecord()** to mark the resource as returned and apply any charges. The **Borrow Record Manager** is passed in as a reference parameter — the **Administrator** neither creates nor owns it, making this a plain association.



3. Borrow Record Manager and Online Resource Manager with Member:

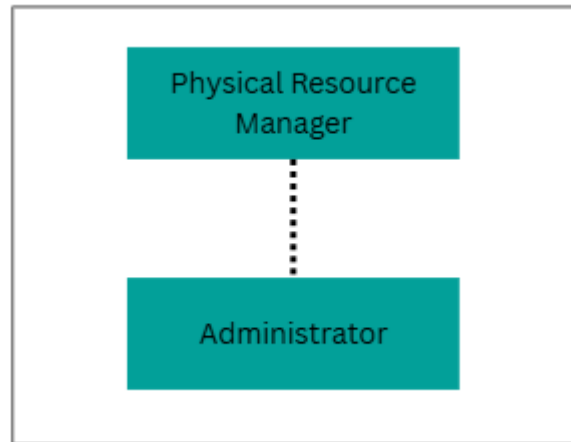
Both **Borrow_Record_Manager** and **Online_Resource_Manager** have plain associations with **Member**. **Borrow Record Manager** uses **Member** implicitly through memberID lookups using the method of Display Borrow Record (memberID).



Online_Resource_Manager is passed as a parameter to **Member::borrowOnlineResource()** and **Member::returnOnlineResource()**, where the member uses the manager to search resources and process refunds. Neither manager owns the **Member** class — they interact through method parameters.

4. Physical Resource Manager and Administrator:

The **Physical_Resource_Manager** is used by **Administrator** to carry out physical resource operations — issuing resources via `physMgr.searchByID()` followed by `resource->markIssued()`, and processing returns via `physMgr.updateResource()`. The manager is passed as a reference parameter into **Administrator**'s methods. This is a directional association marked "**uses**" in the UML, where **Administrator** depends on Physical Resource Manager to complete its tasks but does not own it.





SYSTEM TEST RUNS

- LMS Interface
- Member Interface
- Account Management
- Physical Resources Interface
- Online Resources Interface
- Admin Interface
- Report Generator Interface

SYSTEM OUTPUT:

LMS Interface:

Member Login:

```
----- Member Login -----  
Enter username: fatima_ali  
Enter password: 789567  
| Member 'fatima_ali' logged in. Welcome, Fatima! |
```

Create An Account:

```
Enter username: Abeer_18  
Enter password: nedlife  
Enter first name: Abeer  
Enter last name: Ahmed  
Enter initial balance: 23000  
Enter address: House 345  
---- Account created successfully! ----  
Member ID: 14  
Name: Abeer  
Username: Abeer_18  
Balance: 23000  
Address: House 345
```

Admin Login:

```
Enter admin username: admin  
Enter admin password: admin123  
| Admin User logged in. Welcome, Administrator! |
```

Account Management :

Update Personal Details :

```
Enter new first name: Fatima  
Enter new last name: Ahmed  
Enter new address: House # 407 Main Road  
----- Member updated successfully! -----
```

Update Balance:

```
Enter amount to deposit: 2000  
Balance updated. | Deposit: RS.2000 | New balance: RS.12700.5
```

Update Password:

```
Enter current password : 789567  
Enter the new password(min 6 characters) : 123456  
Confirm new password: 123456  
----- Password updated successfully.-----
```

Admin:

Adding A Physical Resource:

```
----- Add Physical Resource -----  
Title : The Day  
Author : Simon  
Genre : Thriller  
Resource Type : Book  
Publication Date:  
Enter day: 23  
Enter month: 8  
Enter year: 2005  
| Resource with ID PR-18 Added Successfully. |
```

Adding An Online Resource:

```
New Resource ID : OR-16  
Enter Resource Title: The Day  
Enter Author Name: Simon  
Enter Publication Date:  
Enter day: 23  
Enter month: 8  
Enter year: 2056  
Enter Genre: Thriller  
Enter Resource Type: Book
```

Issue Physical Resource:

```
Enter physical resource ID to issue: PR-4  
Enter member ID: 3  
Resource with ID: PR-4 issued successfully to Member ID: 3
```

Process Return Of Physical Resource:

```
Enter borrow ID being returned: 18  
Borrowed Resource has already been returned!
```

Removal Of Physical Resource:

```
Enter physical resource ID to remove: PR-17  
| Resource with ID PR-17 Deleted Successfully. |
```

Removal Of Online Resource:

```
Enter online resource ID to remove: OR-16  
| Resource with ID OR-16 Deleted Successfully. |
```

Report Interface:

Print Customer Borrow History Report :

BORROW ID	MEMBER ID	RESOURCE ID	ISSUE DATE	DUE DATE	RETURN DATE	RETURN STATUS
2	1	PR-1	5/10/2024	15/10/2024	18/10/2024	Returned
4	2	OR-3	8/10/2024	10/10/2024	16/5/2026	Returned
6	3	PR-5	10/10/2024	12/10/2024	20/10/2024	Returned
8	4	OR-7	3/10/2024	5/10/2024	-	Not Returned
10	1	PR-3	12/10/2024	14/10/2024	14/10/2024	Returned
12	5	OR-1	1/10/2024	3/10/2024	-	Not Returned
14	2	PR-2	6/10/2024	8/10/2024	10/10/2024	Returned
16	6	OR-5	9/10/2024	11/10/2024	-	Not Returned
18	3	PR-4	2/10/2024	4/10/2024	5/10/2024	Returned
20	7	OR-9	7/10/2024	9/10/2024	-	Not Returned
22	4	PR-6	11/10/2024	13/10/2024	15/10/2024	Returned
24	8	OR-11	4/10/2024	6/10/2024	-	Not Returned
26	2	OR-3	15/5/2026	17/5/2026	-	Not Returned
28	1	OR-2	15/5/2026	17/5/2026	15/5/2026	Returned
30	1	PR-3	15/5/2026	17/5/2026	-	Not Returned
32	12	OR-8	15/5/2026	29/5/2026	-	Not Returned
34	2	OR-7	16/5/2026	30/5/2026	16/5/2026	Returned
36	2	OR-1	16/5/2026	30/5/2026	-	Not Returned
38	2	PR-1	16/5/2026	18/5/2026	-	Not Returned
40	2	OR-1	16/5/2026	30/5/2026	-	Not Returned
42	2	OR-2	16/5/2026	30/5/2026	16/5/2026	Returned

Print Online Resources Issued Report:

ID	Title	Author	Publication	Available	Type	Genre
OR-1	Programming in Python	Guido van Rossum	1/1/2024	No	Unknown	Technology
OR-3	Machine Learning Basics	Andrew Ng	10/3/2024	No	Unknown	Science

Print Physical Resource Issued Report:

ID	Title	Author	Publication	Genre	Type	Available
PR-1	The Great Gatsby	F. Scott Fitzgerald	15/5/2024	Unknown	Unknown	No
PR-3	1984	George Orwell	10/8/2024	Unknown	Unknown	No
PR-7	Fahrenheit 451	Ray Bradbury	18/6/2024	Unknown	Unknown	No

Physical Resource Interface:

Search Physical Resource By ID:

```
Enter the Resource ID to search: PR-1

----- RESOURCE DETAILS -----
Resource ID      : PR-1
Title           : The Great Gatsby
Author          : F. Scott Fitzgerald
Genre           : Unknown
Resource Type    : Unknown
Available       : No
Publication Date : 15,5,2024
-----
```

Search Physical Resource By Title:

```
Enter Resource Title: 1984

----- RESOURCE DETAILS -----
Resource ID      : PR-3
Title           : 1984
Author          : George Orwell
Genre           : Unknown
Resource Type    : Unknown
Available       : No
Publication Date : 10,8,2024
-----
```

Search Physical Resource By Author:

```
Enter Author Name: George Orwell

----- RESOURCE DETAILS -----
Resource ID      : PR-3
Title           : 1984
Author          : George Orwell
Genre           : Unknown
Resource Type    : Unknown
Available       : No
Publication Date : 10,8,2024
-----
```

Search Physical Resources By Genre:

Enter Genre: Unknown						
ID	Title	Author	Publication	Genre	Type	Available
PR-1	The Great Gatsby	F. Scott Fitzgerald	15/5/2024	Unknown	Unknown	No
PR-3	1984	George Orwell	10/8/2024	Unknown	Unknown	No
PR-4	Pride and Prejudice	Jane Austen	25/2/2024	Unknown	Unknown	Yes
PR-5	The Hobbit	J.R.R. Tolkien	12/7/2023	Unknown	Unknown	Yes
PR-6	The Catcher in the Rye	J.D. Salinger	8/11/2024	Unknown	Unknown	Yes
PR-7	Fahrenheit 451	Ray Bradbury	18/6/2024	Unknown	Unknown	No
PR-8	Jane Eyre	Charlotte Bronte	22/4/2024	Unknown	Unknown	Yes
PR-9	Wuthering Heights	Emily Bronte	30/9/2023	Unknown	Unknown	Yes
PR-10	The Picture of Dorian Gray	Oscar Wilde	14/1/2024	Unknown	Unknown	Yes
PR-11	Moby Dick	Herman Melville	5/12/2024	Unknown	Unknown	Yes
PR-12	War and Peace	Leo Tolstoy	28/3/2024	Unknown	Unknown	Yes
PR-16	Diary Of A Uni Kid	Abeer Ahmed Siddiqui	28/4/2004	Unknown	Book	Yes
PR-18	The Day	Simon	23/8/2005	Unknown	Book	Yes

View Available Physical Resources :

ID	Title	Author	Publication	Genre	Type	Available
PR-4	Pride and Prejudice	Jane Austen	25/2/2024	Unknown	Unknown	Yes
PR-5	The Hobbit	J.R.R. Tolkien	12/7/2023	Unknown	Unknown	Yes
PR-6	The Catcher in the Rye	J.D. Salinger	8/11/2024	Unknown	Unknown	Yes
PR-8	Jane Eyre	Charlotte Bronte	22/4/2024	Unknown	Unknown	Yes
PR-9	Wuthering Heights	Emily Bronte	30/9/2023	Unknown	Unknown	Yes
PR-10	The Picture of Dorian Gray	Oscar Wilde	14/1/2024	Unknown	Unknown	Yes
PR-11	Moby Dick	Herman Melville	5/12/2024	Unknown	Unknown	Yes
PR-12	War and Peace	Leo Tolstoy	28/3/2024	Unknown	Unknown	Yes
PR-13	Discrete Structures	Kenneth H. Rosen	5/3/2000	Technology	Book	Yes
PR-14	Harry Potter	J.K Rowling	26/3/1999	Literature	Book	Yes
PR-15	The Chronicles Of Narnia	C. S. Lewis	28/1/1879	Literature	Book	Yes
PR-16	Diary Of A Uni Kid	Abeer Ahmed Siddiqui	28/4/2004	Unknown	Book	Yes
PR-18	The Day	Simon	23/8/2005	Unknown	Book	Yes

View All Physical Resources:

ID	Title	Author	Publication	Genre	Type	Available
PR-1	The Great Gatsby	F. Scott Fitzgerald	15/5/2024	Unknown	Unknown	No
PR-3	1984	George Orwell	10/8/2024	Unknown	Unknown	No
PR-4	Pride and Prejudice	Jane Austen	25/2/2024	Unknown	Unknown	Yes
PR-5	The Hobbit	J.R.R. Tolkien	12/7/2023	Unknown	Unknown	Yes
PR-6	The Catcher in the Rye	J.D. Salinger	8/11/2024	Unknown	Unknown	Yes
PR-7	Fahrenheit 451	Ray Bradbury	18/6/2024	Unknown	Unknown	No
PR-8	Jane Eyre	Charlotte Bronte	22/4/2024	Unknown	Unknown	Yes
PR-9	Wuthering Heights	Emily Bronte	30/9/2023	Unknown	Unknown	Yes
PR-10	The Picture of Dorian Gray	Oscar Wilde	14/1/2024	Unknown	Unknown	Yes
PR-11	Moby Dick	Herman Melville	5/12/2024	Unknown	Unknown	Yes
PR-12	War and Peace	Leo Tolstoy	28/3/2024	Unknown	Unknown	Yes
PR-13	Discrete Structures	Kenneth H. Rosen	5/3/2000	Technology	Book	Yes
PR-14	Harry Potter	J.K Rowling	26/3/1999	Literature	Book	Yes
PR-15	The Chronicles Of Narnia	C. S. Lewis	28/1/1879	Literature	Book	Yes
PR-16	Diary Of A Uni Kid	Abeer Ahmed Siddiqui	28/4/2004	Unknown	Book	Yes
PR-18	The Day	Simon	23/8/2005	Unknown	Book	Yes

Online Resource Interface:

Borrow Online Resource:

```
Enter the Resource ID: OR-4
Borrowed: Data Science with R successfully! | Borrow ID: 44 | Current borrows: 1
Balance updated. | Charge: RS.1400 | New balance: RS.11300.5
```

Return Online Resource:

```
Enter Borrow ID to return: 44
Balance updated. | Deposit: RS.1400 | New balance: RS.12700.5
Resource returned successfully. | Borrow ID: 44 | Currently Issued Resources are: 0
```

Search Online Resource By ID:

```
Enter the Resource ID to search: OR-3

----- RESOURCE DETAILS -----
ID           : OR-3
Title        : Machine Learning Basics
Author       : Andrew Ng
Genre        : Science
Type         : Unknown
Availability  : No
Publication Date : 10,3,2024
Access Charge : 100/-
-----
```

Search Online Resource By Title:

```
Enter Resource Title: Programming FUNDAMENTALS

----- RESOURCE DETAILS -----
ID           : OR-13
Title        : Programming Fundamentals
Author       : Alexander
Genre        : Science
Type         : Book
Availability  : Yes
Publication Date : 3,6,2018
Access Charge : 100/-
-----
```

Search Online Resource By Author Name:

```
Enter Author Name: Alexander

----- RESOURCE DETAILS -----
ID           : OR-13
Title        : Programming Fundamentals
Author       : Alexander
Genre        : Science
Type         : Book
Availability  : Yes
Publication Date : 3,6,2018
Access Charge : 100/-
-----
```

Search Online Resource By Genre:

Enter Genre: Science						
ID	Title	Author	Publication	Available	Type	Genre
OR-3	Machine Learning Basics	Andrew Ng	10/3/2024	No	Unknown	Science
OR-4	Data Science with R	Hadley Wickham	15/1/2024	Yes	Unknown	Science
OR-13	Programming Fundamentals	Alexander	3/6/2018	Yes	Book	Science

View All Available Online Resources:

ID	Title	Author	Publication	Available	Type	Genre
OR-4	Data Science with R	Hadley Wickham	15/1/2024	Yes	Unknown	Science
OR-5	JavaScript Advanced	Kyle Simpson	8/4/2024	Yes	Unknown	Technology
OR-7	Digital Marketing Essentials	Neil Patel	12/2/2024	Yes	Unknown	Unknown
OR-8	UX Design Principles	Don Norman	18/6/2024	Yes	Unknown	Unknown
OR-9	Cybersecurity Fundamentals	Udacity Staff	3/7/2024	Yes	Unknown	Technology
OR-10	Business Analytics	Edward Tufte	25/3/2024	Yes	Unknown	Unknown
OR-11	Artificial Intelligence Ethics	Timnit Gebru	9/8/2024	Yes	Unknown	Technology
OR-12	Content Marketing Strategy	Gary Vaynerchuk	14/4/2024	Yes	Unknown	Unknown
OR-13	Programming Fundamentals	Alexander	3/6/2018	Yes	Book	Science
OR-15	OOP Workbook	Ms Tehreem Khan	16/5/2026	Yes	Book	Literature

View All Online Resources:

ID	Title	Author	Publication	Available	Type	Genre
OR-1	Programming in Python	Guido van Rossum	1/1/2024	No	Unknown	Technology
OR-3	Machine Learning Basics	Andrew Ng	10/3/2024	No	Unknown	Science
OR-4	Data Science with R	Hadley Wickham	15/1/2024	Yes	Unknown	Science
OR-5	JavaScript Advanced	Kyle Simpson	8/4/2024	Yes	Unknown	Technology
OR-7	Digital Marketing Essentials	Neil Patel	12/2/2024	Yes	Unknown	Unknown
OR-8	UX Design Principles	Don Norman	18/6/2024	Yes	Unknown	Unknown
OR-9	Cybersecurity Fundamentals	Udacity Staff	3/7/2024	Yes	Unknown	Technology
OR-10	Business Analytics	Edward Tufte	25/3/2024	Yes	Unknown	Unknown
OR-11	Artificial Intelligence Ethics	Timnit Gebru	9/8/2024	Yes	Unknown	Technology
OR-12	Content Marketing Strategy	Gary Vaynerchuk	14/4/2024	Yes	Unknown	Unknown
OR-13	Programming Fundamentals	Alexander	3/6/2018	Yes	Book	Science
OR-15	OOP Workbook	Ms Tehreem Khan	16/5/2026	Yes	Book	Literature



PROJECT OUTCOMES

- Learning Outcomes
- Challenges

LEARNING OUTCOMES

- Inheritance makes real sense in practice — shared logic in `User` written once, role-specific stuff only in `Member` / `Administrator`
- Putting validation inside setters (not in menus) means invariants are enforced everywhere automatically.
- Storing objects in a `vector` by value has hidden pitfalls — pointers to elements can go stale after `push_back` reallocates.
- File I/O with `stringstream` + `getline()` for CSV parsing — always check stream state, always match field order exactly.
- A single user action touches multiple subsystems — passing managers by reference and keeping each class focused makes it testable and clean.
- `try/catch` at the interaction boundary is cleaner than inline if-else — let the setter throw, catch it where the user is talking to the program.
- The project also enhanced problem solving skills through implementing real world business logic such as automatic resource ID generation, fine calculation for overdue items, damage charge policies, and availability tracking.
- Made automatic Resource IDs inaccessible to the user and are generated as unique strings for both physical (PR-x) and online resources (OR-x) such that each resource has its own unique ID.
- Improved skills in implementing CRUD operations (Create, Read, Update, Delete).
- Gained experience with conditional logic and string manipulation using `transform()`.
- Implementing multiple search functionalities and formatted display systems improved understanding of efficient data retrieval and user-friendly presentation.
- The implementation process strengthened knowledge of dynamic memory management using pointers and object arrays and preventing memory-related issues such as leaks and dangling pointers.
- The project provided experience with using enumerations (Enums) to represent predefined categories and states in a more structured way .
- Discovered different formats for displaying data in a CLI i.e. (tabular , italic) .

CHALLENGES:

Abeer Ahmed Siddiqui :

- Generating and extracting an automatic ID that was stored as string (OR-x) where x was the id-no which was unique to each resource. Such that it completely erased the possibility of duplicate IDs.
- When dealing with vectors it was important to delete both the resource and the pointer to prevent memory leak thus leading to the use of smart pointers which handled this automatically.
- Saving and loading of the date, which was saved in the CSV file as d,m,y such that it was loaded into the date object `PublicationDate` properly .
- With `PublicationDate` display for tabular display , I had to first convert it into a string then display as d-m-y.

Zehra Baig :

- Coordinating `borrowOnlineResource()` across multiple managers (Online, Borrow, balance) in the exact right order so no side-effect happened before all validations passed.
- `Member::idCount_` static counter incremented on every constructor call including during file loading, causing newly registered members to get wrong IDs
- `updateBorrowRecord()` returns `const BorrowRecord&` but `applyReturnPolicy()` needs a mutable reference — had to make a local copy to fix the compiler error.
- `cin >>` leaves a newline in the buffer so the next `getline()` reads an empty string silently — needed `cin.ignore()` in the right places.
- `updateBalance()` could push balance below zero — had to compute projected balance first and reject if negative.

Horia Younus:

- Designing and implementing CSV-based persistence for borrow records (serialization/deserialization using string streams) was challenging due to strict field ordering, type conversion, and resilience to malformed input.
- Defining the Administrator workflow required designing method signatures that coordinate multiple managers (users, resources, borrow records) while keeping responsibilities well-scoped and avoiding circular dependencies.
- Full-system integration surfaced cross-module issues (signature mismatches, incorrect scope usage, missing/incorrect includes, and forward-declaration decisions); resolving these compilation and linkage problems was time-intensive but essential for a stable build.
- Determining an effective persistence strategy—specifically, where load/save should occur to ensure files are updated immediately after state changes—required careful reasoning about ownership, lifecycle, and consistency.
- Implementing the `lms::run()` application loop required robust input-handling utilities (using `std::numeric_limits` and stream recovery) to prevent invalid input from crashing menus or causing infinite loops.
- Coordinating refactoring and cleanup (removing unnecessary methods) required dependency analysis to ensure features were not indirectly relied upon elsewhere in the system.

Zainab Khan:

- Managing dynamic memory for `PhysicalResource` objects across multiple manager operations was difficult because resources were repeatedly deleted, reloaded, and reallocated. Careful cleanup was required to avoid memory leaks and dangling pointers.
- Generating automatic IDs (PR-x) by extracting numeric values from existing IDs and handling invalid formats safely.
- Implementing case-insensitive searches for title, author, and genre.
- Maintaining proper table formatting because long resource names and author fields could break alignment and reduce readability.
- Correctly converting string values into numeric and date formats.
- Assigning and converting Enum values from user-entered strings was challenging.

CONCLUSION

- Future Expansions
- Conclusion

CONCLUSION

The Library Management System demonstrates a strong and practical implementation of Object-Oriented Programming concepts in a real-world scenario. It effectively applies encapsulation, inheritance, polymorphism, abstraction, and composition to create a well-structured and modular system. The system efficiently manages library resources, member operations, borrowing/returning activities, and transaction records through a user-friendly CLI interface. Overall, this project enhances understanding of OOP design and clearly shows how complex systems can be simplified, organized, and made scalable using classes and objects in C++.

FUTURE EXPANSION

Graphical User Interface:

The Library Management System can be significantly enhanced by integrating a **Graphical User Interface (GUI)**, which would improve overall usability, accessibility, and user interaction compared to the current CLI-based system. A GUI would make navigation more intuitive for users and reduce dependency on command-line inputs.

Data Base Integration :

The system can be further improved by replacing the current **file-based storage mechanism** with a robust database system such as **MySQL or SQLite**. This upgrade would provide better data integrity, faster querying, improved scalability, and more secure and structured data management for large volumes of library records.

Cloud Based Application:

Another important future enhancement is converting the system into a **cloud-based application**, enabling multi-user access from different locations. This would allow librarians, students, and administrators to interact with the system remotely, ensuring real-time synchronization and improved collaboration.

Advanced Searching and Filtering:

The functionality of the system can be expanded by adding an **advanced search and filtering system**, allowing users to search resources based on multiple criteria such as title, author, genre, publication date, and availability status, making resource discovery faster and more efficient.

User Notifications:

A **notification and alert system** can be implemented to automatically remind users about due dates, overdue items, and fine updates. This would help improve resource return rates and reduce late submissions.

Admin Dashboard :

An **admin analytics dashboard** can also be introduced to provide insights such as most borrowed books, active users, overdue statistics, and resource usage trends. This would help administrators make better decisions for resource management.

Elevated Security:

To improve security, the system can include **password encryption and authentication mechanisms**, ensuring that user credentials are securely stored, and access is restricted to authorized users only.

QR Code Integration:

Future versions of the system may also integrate **barcode or QR-code based tracking**, allowing faster issuing and returning of physical resources with automated scanning.

Mobile Application:

Additionally, a **mobile application version** of the system can be developed to provide on-the-go access for users, making it more convenient to browse, borrow, and manage library resources from smartphones.

Overall, these enhancements would transform the current system into a more **modern, scalable, secure, and user-friendly library management solution** suitable for both small and large institutions.



PROJECT CONTRIBUTIONS

- Module Contribution
- Key Implementations
- Developed Components

PROJECT CONTRIBUTIONS:

Abeer Ahmed Siddiqui :

Personal Contribution:

I contributed to the project by designing and implementing the Online Resource Module and its base class LibraryResource . I developed classes to manage online resources and handled file operations for storing and retrieving data. This includes all the methods that allow for smooth addition , ID-generation , removal and searching.

The subsystem contains four primary files :

1. LibraryResource.cpp/.hpp
2. OnlineResource.cpp / .hpp
3. OnlineStorage.cpp/.hpp
4. OnlinerResourceManager.cpp/.hpp.

Components Developed:

Component	Responsibility
LibraryResource	Base Class , that stores Resource title , author name , publication Date , Availability , Genre and Resource Type
OnlineResource	Inherits Library Resource and includes methods for return policy and charging the user for resource usage
OnlineStorage	It manages the connection between storage file and memory through load and save functions
OnlineResourceManager	It manages online resources by adding, removing, displaying and updating resources. It is also responsible for searching resources by their title, author, genre, ID and availability status.

Key Implementation Details:

The LibraryResource class was implemented as a base class to store common details of library items such as title, author, publication date, availability status, genre, and resource type. Encapsulation was achieved using setter and getter functions, while inheritance was used through PhysicalResource and OnlineResource classes to add specialized features like fines and access charges. Enum classes were implemented for genres and resource types to improve organization and type safety, along with conversion functions for easy input handling and display. Resource status was managed using issue and return functions.

The OnlineResource class was implemented to manage digital library resources and their access policies. It includes a fixed access duration using a static constant variable and stores an access charge for each resource. Functions were added to calculate refunds based on the number of unused access days by comparing issue and return dates from the BorrowRecord class. The class also updates

resource availability through the return policy function, demonstrating the use of inheritance, encapsulation, and date-based calculations.

The `OnlineResourceStorage` class was implemented to handle file management for online resources using file handling and smart pointers. It loads resource data from a text file into memory using `std::ifstream`, `stringstream`, and `std::unique_ptr`, while also safely handling corrupted data through exception handling. Resource details such as ID, title, author, availability, genre, type, and publication date are parsed and stored in objects. The class also provides functionality to save all resources back into the file using `std::ofstream`, demonstrating concepts such as dynamic memory management, encapsulation, file persistence, and object serialization.

The `Online_Resource_Manager` class was implemented to manage all operations related to online library resources, including adding, removing, searching, updating, and displaying resources. It uses smart pointers and vectors for dynamic memory management, while file persistence is handled through the `OnlineResourceStorage` class. Unique resource IDs are generated automatically, and user input is validated before storing data. The class supports searching resources by ID, title, author, genre, and availability using case-insensitive comparisons. Tabular and detailed display functions were implemented for better visualization of resource data, and resource updates such as return processing and refund calculation are handled through integration with the `BorrowRecord` and `OnlineResource` classes.

Zehra Baig :

Personal Contribution

My individual contribution to this project was the complete design and implementation of the User Management subsystem. This subsystem is the entry point for every interaction in the LMS — it controls who can log in, what they are allowed to do, and how their data is stored and retrieved between sessions.

The subsystem spans three tightly coupled files:

1. `UserManagement.cpp/.hpp`
2. `UserManager.cpp/.hpp`
3. `MemberStorage.cpp/.hpp`.

Components Developed

Component	Responsibility
User (abstract base)	Core credential fields, getters/setters with validation, password verification and the interactive <code>updatePassword()</code> flow.
Member	Inherits User; adds balance, borrow count, address, and borrow history. Implements <code>borrowOnlineResource()</code> , <code>returnOnlineResource()</code> , <code>updateBalance()</code> , <code>canBorrow()</code> , and <code>createAccount()</code> .
User_Manager	Central session controller: member login, admin login, <code>addMember()</code> , <code>removeMember()</code> , <code>updateMember()</code> , <code>getMemberById()</code> , <code>saveMembers()</code> , and <code>displayAllMembers()</code> .

MemberStorage	CSV-based file persistence: saveAllMembers() serialises the member vector to members.txt; loadAllMembers() deserialises it back on startup.
---------------	---

Key Implementation Details

The User class was designed as an abstract base to enforce a clean separation between shared user attributes and role-specific behaviour. All setter methods include guard clauses that throw `std::invalid_argument` — for example, `setPassword()` rejects any string shorter than six characters, and `setUsername()` rejects empty strings. This ensures that invalid state can never enter the system regardless of where the setters are called.

The Member class manages the full online borrowing lifecycle autonomously. The `borrowOnlineResource()` method follows a strict six-step validation sequence: it verifies the resource ID format, checks the member has not reached the two-resource borrow limit, confirms the member's balance covers the access charge, verifies the resource is currently available, marks it as issued, creates and persists the `BorrowRecord`, and finally deducts the charge from the member's balance. This ordering guarantees that no side-effect occurs unless all preconditions are satisfied.

The Administrator class delegates physical resource issuance to a coordinated call across `User_Manager` (to validate and retrieve the Member), `Physical_Resource_Manager` (to validate and mark the resource), and `Borrow_Record_Manager` (to persist the transaction). After issuance, member state is immediately saved to disk via `userMgr.saveMembers()`.

MemberStorage serialises each Member object as a single comma-separated line containing ID, username, password, first name, last name, address, and balance. On load, each line is parsed using `std::stringstream` with `getline()` and a comma delimiter, and a new Member object is constructed from the extracted fields.

Horia Younus:

Personal Contribution:

My primary contributions in this project were the design and implementation of the complete Borrow Record module as well as the integration of the overall system through the lms controller and application loop. The Borrow Record module serves as the system's central transaction layer for tracking the full borrowing life cycle which ensures that each borrowing transaction is recorded with consistent dates, updated reliably on return and persisted with a CSV-based format for continuity across executions. For the system-integration, I implemented the lms class to coordinate all subsystems within a stable menu-driven runtime flow, standardized the sequence of manager interactions during key operations, resolved cross-module compilation and dependency issues and ensured that state changes have persisted at the appropriate points to maintain a coherent end-to-end workflow across all subsystems.

Components Developed :

Component	Responsibility
LMS (lms.hpp, lms.cpp)	Acts as the system controller and execution coordinator; implements the menu-driven application loop, routes validated user input to the correct subsystem and manages member/admin session flows in a consistent, user-safe manner.
BorrowRecord (BorrowRecord.hpp, BorrowRecord.cpp)	Encapsulates a single borrowing transaction with an auto-generated borrow ID, member ID, resource ID, issue/due/return dates, and return status; provides controlled setters for return-related fields only.
Borrow_Record_Manager (BorrowRecordManager.hpp, BorrowRecordManager.cpp)	Maintains the in-memory borrow record collection, supports transaction creation and return updates, enforces valid return-state transitions, and produces formatted borrow-history reports for members and administrators.
BorrowStorage (BorrowStorage.hpp, BorrowStorage.cpp)	Provides CSV-based persistence for borrow records, including robust parsing and serialization to support reliable loading at startup and saving after each relevant state change.
Administrator (Administrator methods in UserManagement.hpp / UserManagement.cpp)	Implements administrative actions for physical issuance and returns by coordinating User_Manager, Physical_Resource_Manager, and Borrow_Record_Manager in a controlled sequence.

Key Implementation Details:

The `lms::run()` loop was designed as a structured controller that separates interface logic (menus + input) from business logic (manager operations), improving maintainability and reducing coupling between UI prompts and core functionality.

Robust input handling was implemented through reusable helper functions (e.g., numeric reads with stream recovery) to prevent invalid input from breaking the application loop and to ensure predictable menu navigation.

The Borrow Record persistence layer was implemented using CSV and `std::stringstream` parsing, with defensive handling for malformed lines to avoid runtime failures during load operations.

Borrow-record return processing was implemented as a strict state transition: locating the record by borrow ID, preventing duplicate returns, setting return status/date using `Date::today()`, and persisting immediately to keep stored data consistent with in-memory state.

The Administrator workflow was designed to coordinate multiple managers via reference-based calls, ensuring each subsystem remains responsible for its own invariants (member existence and limits, resource availability, and transaction persistence).

System-wide integration required resolving cross-module dependencies (forward declarations vs. includes, signature alignment, and scope correctness) and eliminating redundant/unused methods only after confirming they were not transitively required by other modules, resulting in a stable, compile-consistent final build.

Zainab Khan:

Personal Contribution:

My individual contribution to this project was the complete design and implementation of Physical Resource Management. This subsystem manages the handling of physical library resources by controlling how resources are added, stored, searched, updated, displayed, and maintained throughout the system.

The subsystem spans three closely connected files:

1. `PhysicalResource.cpp/.hpp`
2. `PhysicalStorage.cpp/.hpp`
3. `PhysicalManager.cpp/.hpp`.

Components Developed

COMPONENT	RESPONSIBILITY
Physical Resource	Inherits library resources class and is responsible for calculating fine and damage charges of resources in apply return policy. It implements <code>calculateLateFine()</code> , <code>calculateDamageCharge()</code> and <code>applyReturnPolicy()</code>
PhysicalManager	It manages physical resources by adding, removing, displaying and updating resources. It is also responsible for searching resources by their title, author, genre, ID and availability status.
PhysicalStorage	This component handles all filing and saving physical resources. It implements <code>loadAllPhysicalResources()</code> , <code>saveAllPhysicalResources()</code> .

Key Implementation Details:

Physical Resource contains implementation of applying return policy by calling `calculateLateFine()` and `calculateDamageCharge()` methods that return fine and damage charge if resource is returned overdue days and condition is damaged.

Physical Manager adds resource by loading all resources, creating an object then using setters to set title, author, genre, publication date etc. then at last saving the resource in file. Manager also removes and displays all resources in tabular form. It is also responsible for searching resources by title, author, availability, genre, and ID, and displaying its information in tabular form and in Search by ID , it returns the pointer to that resource for further function overloading.

Physical storage reads(loads) and saves resources in file. This includes parsing CSV records, converting stored values into object attributes, handling invalid or corrupted entries through exception handling.